Sri Lanka Institute of Information Technology



IT3091 – Machine Learning
Year 3, Semester 1s- 2026

Group assignment

Campus: Malabe Campus

Y3.S1. WE. AI. 0101

2026-AI-04

Student ID	Student Name
IT24100853	Gayathri M.G.K
IT24101011	Jayaweera A.J.D
IT24100378	Razni Ahamed M.R
IT24101605	Fernando C.P.H.A.C

12/10/2026

……

Date of Submissions

Github Repository Link : https://github.com/AmalkiFernando/GrainWise.git
(Contains: all notebooks, dataset, decision logs)

Demo Video : https://youtu.be/XcijAbrXGvY?si=crlD46kIow5YLf19

1. Overview

GrainWise asks a practical question for a rice processing mill: can a supplier's declared variety be verified from cheap, image-derived grain measurements, and is a machine-learning model worth deploying to do it?

- We use the UCI Rice (Cammeo and Osmancik) dataset - 3,810 grains, 7 morphological features extracted from grain images (Area, Perimeter, Major_Axis_Length, Minor_Axis_Length, Eccentricity, Convex_Area, Extent), labelled with one of two visually similar varieties. The project builds a binary classifier that outputs a per-grain probability of being Cammeo, compares four modelling approaches trained independently by the four group members against two deliberately simple baselines, and tests whether the added complexity of a tuned model is justified - statistically and economically - before recommending one.

The headline finding: no tuned model beats a single threshold on grain length at a statistically detectable level. We recommend a model anyway, not for accuracy, but because it outputs a tunable probability rather than a fixed label, which lets its decision boundary be moved to match the mill's real cost asymmetry without retraining.

2. Track, lenses, and proposed task/output

Track

- **Guided Data Track** - Code 4 (Agriculture), using the UCI Rice (Cammeo and Osmancik) dataset (DOI 10.24432/C5MW4Z).

We moved to this track after our original Industry Explorer attempt (AgriYield) stalled on dataset-quality issues that made the problem unworkable in the time available. The Guided Data Track let us keep an agriculture-domain framing while working with a clean, well-documented, peer-reviewed dataset, so the group's effort went into analysis and evaluation rigor rather than data cleaning.

Lenses, with reasons

Lens	Role	Why we chose it
Quality-control decision support (primary)	A mill's QC manager needs to decide, per incoming lot, whether a supplier's declared variety matches what the grains' measurements show, before paying a premium price for it.	This framing gives every later choice a concrete owner and a concrete cost: it justifies why we need a probability output and not just a label, why false positives and false negatives are not equally bad, and why threshold tuning (not just accuracy) belongs in the evaluation. It turns an otherwise generic "classify these two classes" exercise into a decision with a stakeholder, a cost, and an action.
Feature-importance analysis (secondary)	Which physical measurements drive the decision - i.e. what an imaging rig must be able to capture reliably.	The dataset's features are highly collinear (VIF up to ~32,500 for Convex_Area), so a naive coefficient or impurity-importance ranking would be misleading on its own. Treating importance as a second lens forced us to check robustness across methods (coefficients, impurity importance, permutation importance) rather than reporting the first ranking we got. which is directly the kind of rigour the rubric rewards and the kind of overclaiming it penalises.

Proposed task/output

- Task: Supervised binary classification. Given a grain's 7 raw morphological measurements (plus engineered scale-invariant ratios), output P(Cammeo) for that grain.
- Decision rule: Threshold that probability against the mill's cost asymmetry (we assume overpaying for a misdeclared lot - an Osmancik grain called Cammeo - is worse than the reverse, and sweep that assumption from 1:1 to 20:1 rather than asserting one fixed number).
- Outputs produced by the project:
    1. A shared, leakage-safe preprocessing/modelling harness (rice_shared.py) used identically by all four models
    2. Four independently trained and tuned models - Logistic Regression, Random Forest, SVM, k-Nearest Neighbours - plus two baselines (majority class, single-feature threshold)
    3. A statistical comparison (corrected paired t-test, McNemar's test) of whether any model's apparent edge is real
    4. A cost-sensitive threshold analysis and one sealed-test-set evaluation
    5. A decision log, workflow diagram, README, and (optionally) a small demo UI, plus each member's own Personal Learning Journey

3. Workflow

We deliberately designed the workflow around one constraint: one shared, versioned harness distributed as code, not as a pre-scaled data file. So that scaling happens inside each model's own cross-validation folds (no leakage) and every member's results are comparable because they ran through identical splits and identical CV folds.

1. Business framing & track selection - agree the QC decision-support framing, the two lenses, and the track before touching data.
2. Load, fingerprint, and split - load the raw .arff, verify no missing values/duplicates, record a SHA-256 fingerprint, and create one stratified 80/20 train/test split (seed 42). The test set is sealed from this point on.
3. Exploratory data analysis - univariate distributions, class balance, correlation matrix, VIF diagnostics on the training partition only.
4. Feature engineering - derive scale-invariant ratios (Aspect_Ratio, Roundness, Solidity) row-wise, pre-split-safe; check for redundancy against existing features (we found Aspect_Ratio is a deterministic transform of Eccentricity).
5. Build and distribute the shared harness (rice_shared.py) - fixed random seed, fixed CV objects, a uniform score() schema, VIF-based feature-set resolution. Distributed as a Python module, not a CSV, so each model fits its own scaler inside its own folds.
6. Parallel model development - each member independently trains and tunes one model through the shared harness:
    - Logistic Regression (Gayathri M.G.K)
    - Random Forest (Fernando C.P.H.A.C)
    - SVM, linear + RBF (Razni Ahamed M.R)
    - k-Nearest Neighbours (Jayaweera A.J.D)
7. Assemble the comparison table - merge all four results_*.csv outputs plus the two baselines into one ranked table.
8. Statistical significance testing - Nadeau–Bengio corrected paired t-tests across all model pairs; McNemar's test against the strongest baseline, to check whether the ranking in step 7 is statistically real.
9. Cost-sensitive threshold tuning - define an explicit (assumed) cost matrix, sweep the decision threshold, and test sensitivity across a range of plausible cost ratios.
10. Final evaluation - fit the single selected model/threshold once on the full training set and evaluate it, once, on the sealed test set.
11. Report assembly - decision log (one shared sheet, Owner column per member), workflow diagram, README, consolidated report.
12. Individual Personal Learning Journeys - each member writes their own, unaided.

Run order for the notebooks: 01 → 02 → (03a–03d in parallel) → 04.

4. Core responsibility of each member

Member	Core responsibility	Model owned
Gayathri M.G.K	Data loading & fingerprinting, EDA, feature engineering, the shared preprocessing/modelling harness (rice_shared.py) that every other member builds on	Logistic Regression
Fernando C.P.H.A.C	Problem/business framing, workflow diagram, decision log coordination	Random Forest
Razni Ahamed M.R	Evaluation design - significance testing (corrected t-test, McNemar), cost-sensitive threshold tuning	SVM
Jayaweera A.J.D	Final recommendation, limitations analysis, report assembly, reproducibility checks	k-Nearest Neighbours

Every member also contributes their own decision-log entries for the choices made while building their model and writes their own Personal Learning Journey independently.

5. Model Performance Comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC (%)	Training Time (s)
Logistic Regression	93.07	92.46	91.31	91.85	97.99	0.0169
Support Vector Machine (SVM)	92.81	92.23	90.93	91.55	97.99	0.9698
K-Nearest Neighbours (KNN)	92.80	91.54	91.69	91.59	97.59	0.0220
Random Forest	92.74	91.90	91.10	91.48	97.82	3.2249
Baseline: Single-Feature Decision Tree	92.90	91.99	91.44	91.68	92.72	0.0126
Baseline: Majority Class	57.22	0.00	0.00	0.00	50.00	0.0030

6. Data dictionary

Raw features

Column	Type	Unit	Description
Area	Integer	pixels²	Number of pixels within the grain's boundary in the image
Perimeter	Float	pixels	Length of the grain's boundary
Major_Axis_Length	Float	pixels	Length of the longest line that can be drawn through the grain (long axis of the fitted ellipse)
Minor_Axis_Length	Float	pixels	Length of the longest line perpendicular to the major axis (short axis of the fitted ellipse)
Eccentricity	Float	unitless (0–1)	Eccentricity of the ellipse fitted to the grain; closer to 1 = more elongated
Convex_Area	Integer	pixels²	Pixel count of the smallest convex hull enclosing the grain
Extent	Float	unitless (0–1)	Ratio of grain area to its bounding-box area
Class	Categorical	—	Target label: Cammeo or Osmancik

Engineered features (derived in rice_shared.add_ratio_features())

Column	Type	Unit	Formula	Description
Aspect_Ratio	Float	unitless	Major_Axis_Length / Minor_Axis_Length	Elongation of the grain. Note: proven to be a deterministic transform of Eccentricity (reconstruction error 5.5×10^{-8}), so it's redundant wherever Eccentricity is already present
Roundness	Float	unitless	$4\pi \times$ Area / Perimeter²	How close the grain's shape is to a perfect circle (1 = circle)
Solidity	Float	unitless	Area / Convex_Area	How "filled in" the grain is relative to its convex hull; lower values indicate a more concave/irregular boundary